README

digital.vasic.containers

Field	Value
Revision	4
Created	2026-04-30
Last modified	2026-10-27 (docs correction after `containers-1.2.1`)
Status	active
Documented state	release `containers-1.2.1` (prepared 2026-10-27; code identical to `containers-1.2.0`), 159 commits after tag `helixcode-v1.1.0`; BREAKING Go API changes since 1.1.x are listed in MIGRATION.md; SemVer deviation: breaking change shipped in 1.2.0/1.2.1 instead of 2.0.0 (operator decision); llmctl `v3.3.0` already consumes the new API, see MIGRATION.md
Change history	CHANGELOG.md
Test coverage	docs/test-coverage.md
Issues	docs/Issues.md (when present)
Continuation	docs/CONTINUATION.md (when present)

A generic, reusable Go module for container orchestration, health checking, lifecycle management, and service discovery. Supports Docker, Podman, and Kubernetes runtimes.

Installation

```
go get digital.vasic.containers
```

Quick Start

```go
package main

import (
    "context"
    "fmt"
    "log"

    "digital.vasic.containers/pkg/boot"
    "digital.vasic.containers/pkg/endpoint"
    "digital.vasic.containers/pkg/health"
    "digital.vasic.containers/pkg/logging"
    "digital.vasic.containers/pkg/runtime"
)

func main() {
    ctx := context.Background()

    // Auto-detect container runtime (Docker or Podman)
    rt, err := runtime.AutoDetect(ctx)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("Using runtime: %s\n", rt.Name())

    // Define service endpoints
    endpoints := map[string]endpoint.ServiceEndpoint{
        "postgres": endpoint.NewEndpoint().
            WithHost("localhost").WithPort("5432").
            WithHealthType("tcp").WithRequired(true).
            WithComposeFile("docker-compose.yml").
            WithServiceName("postgres").
            Build(),
        "redis": endpoint.NewEndpoint().
            WithHost("localhost").WithPort("6379").
            WithHealthType("tcp").WithRequired(true).
            WithComposeFile("docker-compose.yml").
            WithServiceName("redis").
            Build(),
    }

    // Boot all services
    mgr := boot.NewBootManager(endpoints,
        boot.WithRuntime(rt),
```

```
        boot.WithLogger(logging.NewSlogAdapter()),
    )

    summary, err := mgr.BootAll(ctx)
    if err != nil {
        log.Fatalf("Boot failed: %v", err)
    }

    fmt.Printf("Started: %d, Failed: %d\n",
        summary.Started, summary.Failed)
}
```

Features

- **Multi-runtime support**: Docker, Podman, Kubernetes
- **Auto-detection**: Automatically finds available container runtime
- **Health checking**: TCP, HTTP, gRPC, and custom health checks with retry
- **Compose orchestration**: Batch operations grouped by compose file/profile
- **Lifecycle management**: Lazy boot, idle shutdown, concurrency semaphores
- **Resource monitoring**: System and per-container CPU/memory/disk, cluster snapshots
- **Event system**: Publish/subscribe for 20 lifecycle event types
- **Service discovery**: TCP port probe and DNS-based discovery
- **Prometheus metrics**: Built-in metrics collection
- **Pluggable logging**: Bring your own logger (slog adapter included)
- **Remote distribution**: Distribute containers across multiple hosts via SSH
- **Resource-aware scheduling**: 5 strategies (resource_aware, round_robin, affinity, spread, bin_pack)
- **SSH tunnel management**: Cross-host networking with auto port allocation
- **Remote volumes**: SSHFS, NFS, and rsync-based volume sharing
- **Automatic failover**: Detect offline hosts and reschedule containers
- **Environment configuration**: `.env` files and `CONTAINERS_REMOTE_*` env vars
- **Android / Cuttlefish / Genymotion / Apple-simulator / QEMU matrices**: `pkg/emulator`, `pkg/cuttlefish`, `pkg/genymotion`, `pkg/applesim`, `pkg/vm` plus `cmd/emulator-matrix` (with `--build-type` / `--gradle-property`), `emulator-canary`, `vm-matrix`
- **Cross-platform build backends**: `pkg/crossbuild` (host, Linux container, Apple `container` micro-VM on macOS) selected by `crossbuild.Selector`

- **GPU-aware scheduling**: see docs/gpu-scheduling.md
- **Durable remote execution and egress**: `pkg/remoteexec`, `pkg/egress` (SOCKS egress verification)
- **Dependency-ordered orchestrator startup**: topological waves in `pkg/orchestrator`

Configuration and runtime defaults

Remote hosts are configured only through environment variables / a `.env` file (loaded by `pkg/envconfig`); nothing host-specific is compiled in. Names below are taken from `pkg/envconfig/parser.go` and `pkg/envconfig/template.go` (`envconfig.GenerateEnvExample()` prints the full documented template):

Variable	Meaning	Default (code)
`CONTAINERS_REMOTE_ENABLED`	master switch for remote distribution	`false`
`CONTAINERS_REMOTE_DEFAULT_SSH_USER` / `_SSH_KEY` / `_SSH_PASSWORD`	SSH defaults for all hosts	empty
`CONTAINERS_REMOTE_DEFAULT_RUNTIME`	runtime assumed on remote hosts (`docker` / `podman`)	`docker`
`CONTAINERS_REMOTE_SCHEDULER`	`resource_aware`, `round_robin`, `affinity`, `spread`, `bin_pack`	`resource_aware`
`CONTAINERS_REMOTE_HOST_N_NAME`, `_ADDRESS`, `_PORT`, `_USER`, `_KEY`, `_RUNTIME`, `_LABELS`, `_GPU_AUTOPROBE`	per-host settings, `N` = 1, 2, …	per-host; unset falls back to the global defaults above

Example (placeholder values; use your own hostnames and keys, never commit credentials):

```
CONTAINERS_REMOTE_ENABLED=true
CONTAINERS_REMOTE_DEFAULT_RUNTIME=docker
CONTAINERS_REMOTE_HOST_1_NAME=build-host-a
CONTAINERS_REMOTE_HOST_1_ADDRESS=build-a.example.internal
```

```
CONTAINERS_REMOTE_HOST_1_USER=deploy
CONTAINERS_REMOTE_HOST_1_RUNTIME=podman
```

Since `helixcode-v1.1.0`, SET-but-invalid values are reported as errors by `Parse` / `LoadFromFile` instead of silently defaulting (`LoadFromEnv` itself returns a config without validation).

Runtime defaults, stated precisely: - Remote-host runtime default is `docker` (`CONTAINERS_REMOTE_DEFAULT_RUNTIME`). - Local auto-detection (`runtime.AutoDetect`) tries, in order, Podman, Docker, nerdctl, CRI-O, LXD, Kubernetes, and honours `runtime.SetRuntimePriority`. - Podman is supported as a runtime (`PodmanRuntime`, podman-compose shim detection in `pkg/compose`). NOT VERIFIED end-to-end in this documentation pass: no builds or container tests were run for this refresh; rootless Podman against a real remote host and Docker/ Podman behaviour parity on the full matrix are UNCONFIRMED until the containerised full-suite run happens. - Host names such as `thinker` appear in this repository only as illustrative examples (for instance the `ctop --host` usage text and emulator comments); they are not defaults and nothing resolves them implicitly.

Open proposal (not implemented): a first-class local-host build target is described in docs/LOCAL_HOST_BUILD_TARGET_PROPOSAL.md.

Architecture

```
boot.BootManager
├── compose.ComposeOrchestrator  (Docker Compose operations)
├── health.HealthChecker         (TCP/HTTP/gRPC checks)
├── discovery.Discoverer         (Service discovery)
├── distribution.Distributor     (Remote distribution)
├── event.EventBus               (20 lifecycle event types)
├── metrics.MetricsCollector     (Prometheus metrics)
└── logging.Logger               (Pluggable logging)

distribution.Distributor
├── scheduler.Scheduler          (5 placement strategies)
├── remote.HostManager           (Host registry + probing)
├── remote.RemoteExecutor        (SSH command execution)
├── network.TunnelManager        (SSH tunnels)
└── volume.VolumeManager         (SSHFS/NFS/rsync)

lifecycle.LifecycleManager
├── LazyBooter                   (Start on first Acquire)
├── IdleShutdown                 (Stop after inactivity)
└── ConcurrencySemaphore         (Limit parallel users)
```

```
runtime.ContainerRuntime
├── DockerRuntime
├── PodmanRuntime
├── KubernetesRuntime
└── remote.RemoteRuntime         (ContainerRuntime over SSH)
```

Remote Distribution

Distribute containers across local and remote hosts. See docs/REMOTE_DISTRIBUTION.md for the full guide.

```
import (
    "digital.vasic.containers/pkg/distribution"
    "digital.vasic.containers/pkg/envconfig"
    "digital.vasic.containers/pkg/remote"
    "digital.vasic.containers/pkg/scheduler"
)

// Load remote host configuration from .env
cfg, _ := envconfig.LoadFromEnv()
hosts := cfg.ToRemoteHosts()

// Create host manager and register hosts
hm := remote.NewDefaultHostManager(remote.DefaultOptions())
for _, h := range hosts {
    hm.AddHost(h)
}

// Create distributor
dist := distribution.NewDistributor(
    distribution.WithScheduler(
        scheduler.NewDefaultScheduler(hm),
    ),
    distribution.WithHostManager(hm),
    distribution.WithExecutor(
        remote.NewSSHExecutor(remote.DefaultOptions()),
    ),
)

// Distribute containers
summary, _ := dist.Distribute(ctx,
    []scheduler.ContainerRequirements{
        {Name: "web", Image: "nginx:latest"},
        {Name: "cache", Image: "redis:latest"},
    },
)
fmt.Printf("Local: %d, Remote: %d\n",
    summary.LocalContainers, summary.RemoteContainers)
```

Service Orchestrator

Auto-discover and manage all containerized services with automatic remote distribution:

```
import (
    "digital.vasic.containers/pkg/orchestrator"
    "digital.vasic.containers/pkg/compose"
    "digital.vasic.containers/pkg/remote"
)

// Create orchestrator with local compose and optional remote
        support
orch := orchestrator.New(
    orchestrator.WithLocalOrchestrator(composeOrch),
    orchestrator.WithRemoteExecutor(remoteExec),  // optional
    orchestrator.WithHostManager(hostMgr),         // optional
    orchestrator.WithProjectDir("/path/to/project"),
)

// Auto-discover all docker-compose files in docker/ directory
orch.DiscoverServices("docker")

// Or manually add services
orch.AddService(orchestrator.Service{
    Name:        "mcp",
    ComposeFile: "docker/mcp/docker-compose.mcp-servers.yml",
    Description: "MCP servers (32+ servers)",
})

// Start all services (remote if configured, local otherwise)
err := orch.StartAll(ctx)

// Start a specific service
err := orch.StartService(ctx, "mcp")

// List discovered services
services := orch.ListServices()
```

When remote distribution is enabled (both `RemoteExecutor` and `HostManager` provided), all services are automatically deployed to the remote host with automatic fallback to local.

Container Monitoring (ctop)

Real-time container monitoring with top/htop-style display for local and remote containers:

```go
import (
    "context"
    "digital.vasic.containers/pkg/ctop"
    "digital.vasic.containers/pkg/remote"
)

// Create collector with optional remote host support
collector := ctop.NewCollector("podman", hostManager)

// Collect container data
list, _ := collector.Collect(context.Background())
fmt.Printf("Containers: %d running, %d stopped\n", list.Running,
        list.Stopped)

// Create interactive display
display := ctop.NewDisplay(collector, ctop.DefaultDisplayConfig())

// Run interactive TUI (blocks until quit)
display.Run(context.Background())

// Or get a snapshot
snapshot, _ := display.RenderSnapshot(context.Background())
fmt.Println(snapshot)

// Or get JSON output
json, _ := display.RenderJSON(context.Background())
fmt.Println(json)
```

CLI Usage

```bash
# Install the ctop CLI
go install digital.vasic.containers/cmd/ctop@latest

# Run interactive monitoring
ctop

# One-time snapshot
ctop --once

# JSON output
ctop --json

# Filter by host
ctop --host gpu-node-1

# Sort by memory
ctop --sort mem

# Show stopped containers
ctop --all
```

Display Features

- **Color-coded resource usage**: Green (low) → Yellow (medium) → Red (high)
- **Sorting**: CPU, memory, name, state, uptime, runtime, host
- **Filtering**: By host name, container name, running/stopped state
- **Multi-host**: Shows containers from local and remote hosts
- **Remote support**: Integrates with HostManager for distributed monitoring

Anti-Bluff Guarantees (CONST-035, §11.4, round-299)

> Verbatim 2026-05-19 operator mandate (CONST-049 §11.4.17): *"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"*

This repository's PASS bar is "users can use the feature," NOT "tests pass." Every passing test or challenge MUST carry positive runtime evidence captured during execution; metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS is a §11.4 critical defect regardless of how green the summary line looks.

- **CONST-050(B) — 100% test-type coverage.** Unit (mocks allowed only in `*_test.go`), integration (real Docker/Podman), e2e (real SSH targets), security, stress, benchmark, plus 12 challenges under challenges/scripts/. Per-symbol ledger lives at docs/test-coverage.md.
- **Paired-mutation discipline (§1.1).** Every gate has a paired mutation that deliberately breaks the production code path and asserts the gate fails. A gate that survives mutation is a bluff gate. The round-299 paired-mutation script `challenges/scripts/containers_describe_challenge.sh` ships with both `--mutate` mode (exit 99 = mutation witnessed) and a normal mode (exit 0 = all five conditions PASS).
- **Remote distribution — CONST-045 .env-driven only.** Host configuration lives exclusively in `.env` via `CONTAINERS_REMOTE_HOST_N_*` env vars (loaded by `pkg/envconfig`). NO

hostname / IP / SSH user / key path is hardcoded in any source / test / challenge. When `CONTAINERS_REMOTE_ENABLED=false`, remote-touching tests emit `SKIP-OK:` markers per CONST-045 and exit 0 (skip is not failure, but skip is loud).

- **No-fakes-beyond-unit-tests (CONST-050(A)).** Production code under `pkg/`, `cmd/`, `internal/buildpkg/` MUST NOT import from any `internal/mocks/` path. Mocks / stubs / `TODO` / `FIXME` / "for now" / "in production this would" patterns exist ONLY inside `*_test.go`.
- **i18n / CONST-046 — no hardcoded human-readable strings.** User-facing text is loaded from `pkg/i18n/bundles/active.<locale>.yaml`. Round-299 added 5 locales beyond English (fr / de / ja / sr / zh); the paired-mutation challenge asserts all 6 bundles present + non-empty and exits 99 when any bundle is removed.

Run the round-299 challenge locally

```
# Normal mode — must exit 0; emits PASS for every condition
bash challenges/scripts/containers_describe_challenge.sh

# Paired-mutation mode — must exit 99; restores the working tree
        on EXIT trap
bash challenges/scripts/containers_describe_challenge.sh --mutate
```

The challenge respects the host-power-management hard ban (CONST-033 / §12), performs no sudo / suspend / hibernate / poweroff calls, never echoes secrets (§11.4.10), and runs in O(seconds) without any container start.

Android Emulator (pkg/emulator)

`pkg/emulator` provides multi-target Android emulator orchestration satisfying parent project's clauses 6.I (Multi-Emulator Container Matrix), 6.K (Builds-Inside-Containers Mandate), 6.V (Container Emulators Mandate), and 6.X (Container-Submodule Emulator Wiring Mandate).

Emulator variants

Host OS	Acceleration	Runner	Gate-eligible
Linux x86_64	KVM (`/dev/kvm`)	`containerized`	√
	HVF (host-only API)	`host-direct`	√

Host OS	Acceleration	Runner	Gate-eligible
macOS arm64			
Windows	WHPX (host-only API)	`host-direct`	✓

`AccelProfileForOS(runtime.GOOS)` returns the deterministic OS-correct profile. `ResolveRunner("auto", runtime.GOOS)` picks the OS-correct runner. See `pkg/emulator/accel.go` for the complete rationale.

Boot reliability features

- **Pre-boot ADB hygiene** (added 2026-06-03): `Boot()` calls `ResetADBHygiene()` before launching the emulator. This disconnects any phantom TCP entries (e.g. `localhost:5555 offline`) that wedge adb's device tracking, then cycles `adb kill-server` / `adb start-server` so the daemon starts from a clean state. Forensic anchor: a phantom `localhost:5555 offline` entry caused `discoverNewSerial` to never observe the new emulator, producing boot-timeout on every attempt even after the emulator had started.
- **Pre-boot zombie cleanup**: `runOne` calls `Cleanup()` before every emulator boot to remove stale `qemu-system-*` processes left by interrupted runs. This prevents ADB-port collisions on multi-AVD matrix iterations.
- **AVD lock clearing**: `clearAVDLock(avdName)` removes `$HOME/.android/avd/<name>.avd/<name>.lock` before boot so an unclean previous shutdown cannot block the next boot.
- **Configurable boot timeout**: `MatrixConfig.BootTimeout` / `--boot-timeout` flag is honored on both the host-direct and containerized paths.
- **Reap on boot-timeout** (added 2026-06-03): when `Boot()` port-discovery times out, `Cleanup()` reaps any orphaned `qemu-system-*` processes to prevent hot zombies consuming CPU/RAM and holding AVD lock files. When `WaitForBoot()` times out, `KillByPort(consolePort)` terminates the specific emulator by its console-port argv token; if that matches zero processes, `Cleanup()` is the fallback. Forensic anchor: an unreapped emulator was observed at ~370% CPU after a matrix FAIL row, causing the next boot to also time out (CPU starvation).
- **Offline transport classification** (added 2026-06-03): `ParseADBDevicesLine` classifies `adb devices` lines into `ADBStateDevice`, `ADBStateOffline`, `ADBStatePhantomTCP`, and

`ADBStateUnauthorized`. TCP endpoints in offline state (`ADBStatePhantomTCP`) are explicitly disconnected by `ResetADBHygiene` before kill-server so they cannot re-wedge the daemon immediately after restart.

- **Boot diagnostic capture** (added 2026-06-03): on `Boot()` or `WaitForBoot()` timeout, `CaptureBootDiagnostic()` captures `adb devices` output and a `getprop` snapshot into a `boot-diag-<ts>.json` file in the per-AVD evidence directory. The `BootResult.BootDiag` field (non-nil only on error paths) surfaces this diagnostic to the matrix runner for embedding in the attestation row — satisfying clause 6.I Group-B `diag` intent.
- **Reliable teardown**: `Teardown` polls `adb devices` until the emulator transitions out of "device" state before returning, preventing the next boot from colliding with a still-running emulator. `KillByPort` provides a port-strict SIGTERM/SIGKILL fast-path for stuck processes.

Release cold-start canary (`cmd/emulator-canary`)

The canary closes the §6.Z tooling gap: debug-signed `androidTest` APKs cannot test release-signed APKs (signature mismatch), so the canary uses `adb shell am start` + logcat to exercise the release build directly.

```
emulator-canary \
  --android-sdk-root $ANDROID_SDK_ROOT \
  --apk releases/1.2.36/android-release/app-release.apk \
  --package digital.vasic.app.client \
  --activity .MainActivity \
  --avd Pixel_8:35:phone \
  --evidence-dir .ci-evidence/canary-1.2.36 \
  --cold-boot
```

Exit codes: - `0` — activity reached RESUMED state AND no FATAL in logcat (canary PASS) - `1` — activity did not resume OR FATAL crash detected (canary FAIL) - `2` — configuration error

The canary writes a `canary-attestation.json` and `logcat.txt` under `--evidence-dir`. The primary anti-bluff assertion is on user-visible state: "activity resumed AND logcat FATAL-free" — NOT "APK installed without error".

Matrix runner (`cmd/emulator-matrix`)

```
emulator-matrix \
  --android-sdk-root /opt/android-sdk \
  --apk releases/1.2.36/android-debug/app-debug.apk \
```

```
  --test-class
        app.example.challenges.Challenge01AppLaunchAndTrackerSelectionTest
        \
  --evidence-dir .ci-evidence/1.2.36 \
  --avds Pixel_8:35:phone,Pixel_Tablet:34:tablet \
  --runner auto \
  --cold-boot
```

`--runner auto` selects the OS-correct runner: `containerized` on Linux (requires `--container-image`), `host-direct` on macOS/Windows.

OTA Device Emulator (images/ota-device-emu + cmd/ota-device-emu-boot)

`images/ota-device-emu/` provides a podman/docker **OTA device-emulator runtime image**: a minimal Alpine + CA-certs base that runs a consumer-supplied static `ota-device-emu` Go binary impersonating one OTA target device against an OTA control plane — for **Tier-1 control-plane protocol testing in place of live hardware**. The binary is built in the consuming project and supplied via volume mount or a derived image (see the env-var contract in `images/ota-device-emu/entrypoint.sh`).

`cmd/ota-device-emu-boot` is the on-demand bring-up recipe: it composes `pkg/boot` + `pkg/health` to start a control-plane + emulator stack and health-check it (HTTP health for the control plane; runtime `running`-state for the emulator). No manual `compose up` (Constitution §11.4.76).

Honest boundary: this is Tier-1 (control-plane protocol) only. The real Android A/B `update_engine` + AVB/dm-verity apply flow needs Cuttlefish-on-Linux-KVM and is NOT this image's job. Full design: docs/ota-device-emulation.md.

```
podman build -f images/ota-device-emu/Dockerfile \
  -t ota-device-emu:dev images/ota-device-emu/
podman run --rm ota-device-emu:dev --help
go run ./cmd/ota-device-emu-boot \
  --compose images/ota-device-emu/docker-compose.example.yml
```

License

MIT